159.102 Notes

Section 10 : Data Structures, Vectors and Stacks

What is a Data Structure?

A data structure is a collection of data items, organised in a particular way. They are used to store large amounts of data in a computer and usually incorporate mechanisms for the efficient location and storage of individual data items.

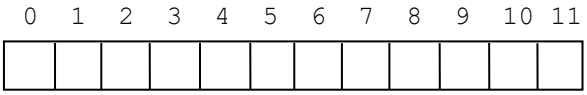
Our first Data Structure – the array

Arrays are probably the simplest type of data structure. They provide the first step towards storing large amounts of data. Arrays were introduced in 159.101 and we do some quick revision here:

Here is an example of an array in a C++ program:

```
float rain[12];
```

The number 12 informs the system how many memory locations are required. In this example, the system would set aside 12 memory locations like this:



All these locations have the same name `rain`

We use the **index** to identify which **array element** we are working with. The index is an integer value. For example, if we want to store 15.7 in the third element, then the C++ code would be:

```
rain[2] = 15.7;
```

Note that the first element in the array always has index 0.

Advantages and Disadvantages of arrays

Programmers always have to make choices when designing a new program (e.g. "should I use a for-loop or a while-loop?"). In order to make the right choices, it is important to be well informed. Programmers have a choice about which data structure to use in a program. In order to select the best data structure for the current program, you need to be able to compare different data structures. We therefore discuss the advantages and disadvantages of each data structure. There is a lot more about this in the course 159.201.

Advantages of arrays

- Arrays are simple and easy to understand
- The fastest data structure – computers are designed to work with arrays
- Direct access (also called random access)**

Disadvantages of arrays

- Fixed size (the number in the square brackets)
- Elements can be only one data type (e.g. float)

** Direct access – the ability to directly look at any element. As long as we know the index, we can look directly at any element of the array.

Discussing the Advantages and Disadvantages of Arrays

People often discuss the disadvantages of arrays. For example, they will focus on the fact that an array has to be a fixed size. However, most of the time, this is easily solved by making an array far bigger than you actually need it. The reason for this is that, if you need to store more items than expected, the total number of items will still be less than the maximum allocated for the array.

Example: Write a program to store test results for all students in a class. There are never more than 100 students in the class. When you design your program, set the size of the array to be 2000, i.e. much bigger than what is required. It is quite likely that when your program is working, they will say "Oh dear, we just realised that the biggest class is actually 250 students." If you had originally designed your program with a maximum of 100 then you have a problem. However, if you had set the maximum to be 2,000 then no problem at all.

As long as you follow the guideline "always make an array much bigger than you need", fixed size will not be a problem. It only becomes a problem if your array is so huge that you start to run out of memory.

The second disadvantage of arrays ("Elements can only be one data type") can be solved by creating classes and making an array of objects. Each element of the array is thus an object and can store whatever data items you define within the object. Note that you can also avoid having to use "corresponding arrays" (as taught in 159.101) by creating an array of objects. For example:

```
#include <iostream>
using namespace std;

class student_class {
  int id;
  string name;
  int result1, result2;
};

student_class student[2000];
```

In this example, we (a) can store several data items in one element of the array, (b) avoid using corresponding arrays, and (c) make the array much bigger than we need.

(*Note: if you do not remember corresponding arrays from 159.101, that is ok, you do not need to know about them for 159.102*)

People talk less about the advantages of arrays, in particular "arrays are simple". When you have a choice to make during the design of a program, always choose the simplest option (if possible). The complicated option is much more likely to lead to errors later on.

Vector

A vector is an array that can resize itself automatically as data is inserted or deleted (also known as a **dynamic array**). Vector is a class provided in the **Standard Template Library** (STL). A template is a data structure where the data type is not defined. The actual data type (or class) is only defined when you create the data structure. The STL is a collection of libraries providing a range of data structures.

Here is an example program that uses a vector:

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> data;

int main() {
  int i;
  data.push_back(2);
  data.push_back(4);
  data.push_back(6);
  for (i = 0; i < data.size(); i++) {
    cout << data[i] << ", ";
  }
  cout << endl;
}
```

Important points about this program:

- it includes the `vector` library
- it creates a vector of integer values
- you have to specify the data type `<int>` when creating the vector
- the vector does not have a fixed size – there are no square brackets
- you can insert an item at the back of the vector using `push_back`
- use the method `size()` to find out how many items are in the vector

You can use an index with a vector (e.g. `data[i]` in the example). HOWEVER you can only use the index AFTER data items have been loaded into the vector.

The vector class

The vector class provides you with variables and methods. You should know the basic methods associated with a vector. These are:
`push_back()` – insert a new item at the back of the vector
`pop_back()` – delete the item at the back of the vector
`size()` – returns the number of items in the vector

There are many other methods available with vectors but you do not need to know them for our course. You may use any of them if you want to. The full description of the vector class is here:
https://www.cplusplus.com/reference/vector/vector/

Advantages and Disadvantages of vectors

Advantages of vectors

- No fixed size – store as much (or as little) as you want

Disadvantages of vectors

- More complicated to use than arrays
- Significantly slower than arrays

- Elements can be only one data type (e.g. float) (same as arrays)

Many people get overexcited about the one advantage of vectors (no fixed size) without considering the disadvantages. Vectors are significantly slower than arrays because behind the scenes, the computer is doing a lot of extra work. When a vector is created, it is actually a fixed size (just like an array) but the system does not inform you what that size is (actually one of the methods can do this). When the vector gets full, the computer has to allocate another section of memory, keep track of how many sections the vector is taking up, and load the new data item into the new section of memory. All this takes up extra time and slows the program down.

When you are designing a new program and you realise that you need an array (or vector) you will need to make a choice. For example, you could say: "Arrays are fixed size but I just need to make the array much bigger than what is required plus arrays are faster and simpler to use." Or you could say "People will be really impressed if I use vectors." This is entirely up to you.

See also the discussion at https://www.geeksforgeeks.org/cpp/when-to-use-vector-instead-of-array-in-cpp/ comparing vectors and arrays. Functions can return a vector, but not an array; the Standard Template Library (STL) contains many methods and algorithsm (e.g. for sorting) that can operate on vectors.

Stack

Imagine a stack of plates. Each plate is placed on top of the previous plate. The only plate you can see is the top one, all the other plates are hidden in the stack beneath the top plate. The only plate you can remove from the stack is the top one (well, you shouldn't try to remove any of the plates that are further down the stack). For more information about stacks (and to see the stack of plates): https://en.wikipedia.org/wiki/Stack_(abstract_data_type)
*Note: you **do not** need to know all the extra information provided in this Wikipedia article.*

A stack is a LIFO data structure. LIFO means "Last In, First Out". All the action in a stack takes place at one end called 'top'. When an item is inserted into the stack it is placed at the top and the new item becomes the top item. The only item that can be removed from the stack is the top item. If you want to find an item in the stack, the only way to do it is to keep removing items from the top until the item you want appears as the top item.

Stack is a class provided in the **Standard Template Library** (STL). A template is a data structure where the data type is not defined. The actual data type (or class) is only defined when you create the data structure. The STL is a collection of libraries providing a range of data structures.

Here is an example program that uses a stack:

```
#include <iostream>
#include <stack>
using namespace std;

void displayandempty();

stack<char> mystack;

int main() {
  char ch;
  while (ch != '?') {
    cout << "Enter a character (? to stop) ";
    cin >> ch;
```

```
    if (ch != '?') {
      mystack.push(ch);
    }
  }
  displayandempty();
}

void displayandempty() {
  char k;
  cout << "The data in reverse order:\n";
  while (mystack.empty() != true) {
    k = mystack.top();
    cout << k << ", ";
    mystack.pop();
  }
  cout << endl;
  cout << "All items have now been removed from the stack\n";
}
```

Important points about this program:

- it includes the `stack` library
- it creates a stack of characters – you can create a stack of any data type or class
- you have to specify the data type `<char>` when creating the stack
- the stack does not have a fixed size – there are no square brackets
- the method `push` is used to insert a new item at the top of the stack
- the method `pop` is used to remove the top item from the stack
- note how to use the method `top`, this is the only way to get data out of a stack
- when you `pop` all items of the stack, you empty the stack

The stack class

The stack class provides you with variables and methods. You should know the basic methods associated with a stack. These are:
`push()` – insert a new item at the top of the stack
`pop()` – delete the item at the top of the stack
`top()` – get the item at the top of the stack
`empty()` – returns true if the stack is empty (there are no items in the stack)
`size()` – returns the number of items in the stack

There are other methods available with stacks but you do not need to know them for our course. You may use any of them if you want to. The full description of the stack class is here:
https://www.geeksforgeeks.org/stack-in-cpp-stl/

How to implement a stack (without the stack library)

A stack is usually implemented by using a vector (or an array) by doing something like this (although some details may be missing):

```
class stack_class {
// WARNING: this stack cannot hold more than 1000 items
private:
```

```
  int total;
  int items[1000];
public:
  void push(int num);
  // include some other methods here
};
```

The push method would then be something like this (more details may be required):

```
void stack_class::push(int num) {
  items[total] = num;
  total++;
}
```

When to use a stack

Stacks are used in various ways inside operating systems, for example stack-based memory allocation. For more information (this is not part of our course) see here:
https://en.wikipedia.org/wiki/Stack-based_memory_allocation

We also use stacks in our own programs if we want to reverse the order of a sequence of data items. For example, the program to convert a decimal number into a binary number requires the order of digits to be reversed, as shown in the example program below:

```
#include <iostream>
#include <stack>
using namespace std;

stack<int> digits;

int main() {
  int num;
  cout << "Enter a decimal number ";
  cin >> num;
  while (num > 0) {
    digits.push(num % 2);
    num = num / 2;
  }
  cout << "The binary number is ";
  while (digits.empty() != true) {
    cout << digits.top();
    digits.pop();
  }
  cout << endl;
}
```

Using the memory stack to keep track of recursion

The system maintains a stack of blocks of memory which is what allows programmers to be able to use recursion. This stack is used to keep information from previous function calls. Each function call stores a block of information including current values of local variables, next instruction to be executed (after the function call), etc. We will now repeat the walkthrough that we did before when

looking at the recursive function to calculate factorial. We will note when the memory stack is used in the process.

```
int factorial(int x) {
  int result;
  if (x == 0) {
    return 1;
  }
  result = x * factorial(x - 1);
  return result;
}
```

Let us assume we want to calculate the factorial of 3

main: calls the factorial function with x = 3

call 1: The function is called with x = 3
The function calculates: result = 3 * factorial(2)
Information for function call 1 is pushed onto the stack

call 2: The function is called with x = 2
The function calculates: result = 2 * factorial(1)
Information for function call 2 is pushed onto the stack

call 3: The function is called with x = 1
The function calculates: result = 1 * factorial(0)
Information for function call 3 is pushed onto the stack

call 4: The function is called with x = 0
The function returns the value of 1 (the base case)

so…

Pop information for one function call off the stack.
Using this information takes us back to call 3 (plus the result from call 4)
Function call 3 can now continue, calculate a result of 1 * 1 and return 1

Pop information for one function call off the stack.
Using this information takes us back to call 2 (plus the result from call 3)
Function call 2 can now continue, calculate a result of 2 * 1 and return 2

Pop information for one function call off the stack.
Using this information takes us back to call 1 (plus the result from call 2)
Function call 1 can now continue, calculate a result of 3 * 2 and return 6 to main

Note: for purposes of our course, we simply want to have the basic idea that the system uses the memory stack to make recursion work – this is why too many recursive function calls can result in the error “stack overflow”. This means that the stack is full and no more recursive calls are possible.

Complexity of algorithms and the "Big-O" notation

One of the most famous computer science books of all time is Niklaus Wirth's "Algorithms + Data Structures = Programs" from 1976. (Read it here.) Its title emphasizes how the choice of a data structure and an algorithm are both very important and are closely related. They affect the running time of a program, which can be critical, especially for large data sets. The correct choice can mean the difference between a runtime of milliseconds and months.

For example, if an array of numbers is stored and maintained in sorted order, then very fast searching algorithms (e.g. binary search) are possible.

Usually the exact runtime of an algorithm is not important. What counts is how rapidly it increases for large datasets.

Consider the task of sorting N numbers. We have two candidate algorithms, A and B. Algorithm A uses 0.2 N^2 comparisons, while algorithm C uses 5 N $\log_{10}$ N comparisons. Here is a table of the number of comparisons each algorithm uses for values of N from 10 to 10,000

N	A	B
10	20	50
100	2,000	1,000
1,000	200,000	15,000
10,000	20,000,000	200,000

Although algorithm A is faster for the small values of N, for larger values, algorithm B is faster – and the improvement gets more and more extreme as N increases.

The constants 0.2 and 5 don't affect the growth rate of these functions, so from now on we will ignore them.

We say algorithm A is $O(N^2)$ (or "has time complexity $O(N^2)$"), and algorithm B is O(N log N). This is the "Big-O" notation.

The following table compares the growth rates of some other functions, from slowest to fastest growth.

N	log N	N	N log N	N^2	N^3	N!
10	1	10	10	10^2	10^3	3.7×10^6
100	2	100	200	10^4	10^6	10^{158}
1,000	3	1,000	3,000	10^6	10^9	10^{2568}
10,000	4	10,000	40,000	10^8	10^{12}	10^{35659}

An O(log N) algorithm, such as binary search, is blindingly fast even for enormous problems.

Amongst famous sorting algorithms, bubble sort and exchange sort are $O(N^2)$, while quicksort is O(N log N) so it will be much, much faster. Bubble sort takes many minutes to sort a million numbers while quicksort will do it instantly.

The "travelling salesman" is a famous computer science problem for which all known algorithms are slow. The task is to find the shortest path visiting N cities. There are N! (= 1×2×3×…×N) possible paths, one for each possible order of visiting the cities, so checking them all takes time O(N!) and is impractical once N is larger than 10 or 12.

Over time, computers have become faster and faster. This leads to a demand for tackling larger and larger problems – for example, searching the whole internet billions of times a day. This means that the choice of algorithm has become more and more important.

The study of the time and memory used by different algorithms, how they work in different situations, and what are the best possible algorithms, is called "computational complexity." It is an important branch of computer science.

Example. What is the complexity of the following algorithm?

```
int main(){
  int i, j, k, sum = 0, N = 100;
  for (i = 1,; i <= N; i++){
    for (j = 1,; j <= N; j++) {
      for (k = 1,; k <= N; k++) {
        sum = sum + i*j*k;
      }
    }
  }
}
```

Answer. The line '`sum = …`' is executed N^3 times. Each requires 2 multiplications and an addition. So there are 3 N^3 maths operations altogether. There are some other operations involved in the loops, but we ignore these as they take less time. We also ignore the constant 3. **The algorithm is $O(N^3)$.**

Is there a faster algorithm for the same calculation? Yes, there is. Using some algebra and the ∑ notation for sums (don't worry if you haven't seen this before, this is for illustration only), the calculation can be simplified as follows:

$$\sum_{i=1}^{N}\sum_{j=1}^{N}\sum_{k=1}^{N} ijk = \left(\sum_{i=1}^{N} i\right)\left(\sum_{i=1}^{N} j\right)\left(\sum_{i=1}^{N} k\right) = \left(\sum_{i=1}^{N} i\right)^3 = \left(\frac{1}{2}N(N+1)\right)^3$$

(See this link for an explanation of the sum of the first N integers.) This means we can use the following equivalent algorithm:

```
int main(){
  int sum, N = 100;
  sum = N*N*N*(N+1)*(N+1)*(N+1)/8;
}
```

which has no loops at all. It takes 9 operations regardless of the value of N. We have reduced an $O(N^3)$ algorithm to an O(1) algorithm – a massive speedup.

More information about Data Structures

This is NOT part of our course.

A good working knowledge of data structures is vital for all professional programmers. If you can fit it into your degree, it is a really good idea to study 159.201 at Massey. 159.201 covers the full range of data structures in detail. A useful overview of data structures can be found here: https://towardsdatascience.com/8-common-data-structures-every-programmer-must-know-171acf6a1a42